

UNIT38: PHYSICALLY-BASED RENDERING

【学習要項】

- PBR : Physically-based rendering
- BRDF : Bidirectional reflectance distribution function

【演習手順】

1. シェーダーの実装 (変更)

① HLSL ヘッダーファイルの追加 (bidirectional_reflectance_distribution_function.hlsl)

```
1: #ifndef __BIDIRECTIONAL_REFLECTANCE_DISTRIBUTION_FUNCTION_HLSL__
2: #define __BIDIRECTIONAL_REFLECTANCE_DISTRIBUTION_FUNCTION_HLSL__
3:
4: float3 f_schlick(float3 f0, float3 f90, float VoH)
5: {
6:     return f0 + (f90 - f0) * pow(clamp(1.0 - VoH, 0.0, 1.0), 5.0);
7: }
8: float v_ggx(float NoL, float NoV, float alpha_roughness)
9: {
10:     float alpha_roughness_sq = alpha_roughness * alpha_roughness;
11:
12:     float ggxv = NoL * sqrt(NoV * NoV * (1.0 - alpha_roughness_sq) + alpha_roughness_sq);
13:     float ggxl = NoV * sqrt(NoL * NoL * (1.0 - alpha_roughness_sq) + alpha_roughness_sq);
14:
15:     float ggx = ggxv + ggxl;
16:     return (ggx > 0.0) ? 0.5 / ggx : 0.0;
17: }
18: float d_ggx(float NoH, float alpha_roughness)
19: {
20:     const float PI = 3.14159265358979;
21:     float alpha_roughness_sq = alpha_roughness * alpha_roughness;
22:     float f = (NoH * NoH) * (alpha_roughness_sq - 1.0) + 1.0;
23:     return alpha_roughness_sq / (PI * f * f);
24: }
25: float3 brdf_lambertian(float3 f0, float3 f90, float3 diffuse_color, float VoH)
26: {
27:     const float PI = 3.14159265358979;
28:     return (1.0 - f_schlick(f0, f90, VoH)) * (diffuse_color / PI);
29: }
30: float3 brdf_specular_ggx(float3 f0, float3 f90, float alpha_roughness,
31:     float VoH, float NoL, float NoV, float NoH)
32: {
33:     float3 F = f_schlick(f0, f90, VoH);
34:     float Vis = v_ggx(NoL, NoV, alpha_roughness);
35:     float D = d_ggx(NoH, alpha_roughness);
36:
37:     return F * Vis * D;
38: }
39:
40: #endif
```

② ピクセルシェーダー (gltf_model_ps.hlsl) で bidirectional_reflectance_distribution_function.hlsl をインクルードする

③ ピクセルシェーダー (gltf_model_ps.hlsl) の main 関数を変更する

```
1: float4 main(VS_OUT pin) : SV_TARGET
2: {
3:     const float GAMMA = 2.2;
4:
5:     const material_constants m = materials[material];
6:
7:     float4 basecolor_factor = m.pbr_metallic_roughness.basecolor_factor;
8:     const int basecolor_texture = m.pbr_metallic_roughness.basecolor_texture.index;
9:     if (basecolor_texture > -1)
10:     {
11:         float4 sampled = material_textures[BASECOLOR_TEXTURE].Sample(sampler_states[ANISOTROPIC], pin.texcoord);
12:         sampled.rgb = pow(sampled.rgb, GAMMA);
```

UNIT38: PHYSICALLY-BASED RENDERING

```
13:     basecolor_factor *= sampled;
14: }
15:
16: float3 emissive_factor = m.emissive_factor;
17: const int emissive_texture = m.emissive_texture.index;
18: if (emissive_texture > -1)
19: {
20:     float4 sampled = material_textures[EMISSIVE_TEXTURE].Sample(sampler_states[ANISOTROPIC], pin.texcoord);
21:     sampled.rgb = pow(sampled.rgb, GAMMA);
22:     emissive_factor *= sampled.rgb;
23: }
24:
25: float roughness_factor = m.pbr_metallic_roughness.roughness_factor;
26: float metallic_factor = m.pbr_metallic_roughness.metallic_factor;
27: const int metallic_roughness_texture = m.pbr_metallic_roughness.metallic_roughness_texture.index;
28: if (metallic_roughness_texture > -1)
29: {
30:     float4 sampled = material_textures[METALLIC_ROUGHNESS_TEXTURE].Sample(sampler_states[LINEAR],
31:         pin.texcoord);
32:     roughness_factor *= sampled.g;
33:     metallic_factor *= sampled.b;
34: }
35:
36: float occlusion_factor = 1.0;
37: const int occlusion_texture = m.occlusion_texture.index;
38: if (occlusion_texture > -1)
39: {
40:     float4 sampled = material_textures[OCCLUSION_TEXTURE].Sample(sampler_states[LINEAR], pin.texcoord);
41:     occlusion_factor *= sampled.r;
42: }
43: const float occlusion_strength = m.occlusion_texture.strength;
44:
45: const float3 f0 = lerp(0.04, basecolor_factor.rgb, metallic_factor);
46: const float3 f90 = 1.0;
47: const float alpha_roughness = roughness_factor * roughness_factor;
48: const float3 c_diff = lerp(basecolor_factor.rgb, 0.0, metallic_factor);
49:
50: const float3 P = pin.w_position.xyz;
51: const float3 V = normalize(camera_position.xyz - pin.w_position.xyz);
52:
53: float3 N = normalize(pin.w_normal.xyz);
54: float3 T = has_tangent ? normalize(pin.w_tangent.xyz) : float3(1, 0, 0);
55: float sigma = has_tangent ? pin.w_tangent.w : 1.0;
56: T = normalize(T - N * dot(N, T));
57: float3 B = normalize(cross(N, T) * sigma);
58:
59: const int normal_texture = m.normal_texture.index;
60: if (normal_texture > -1)
61: {
62:     float4 sampled = material_textures[NORMAL_TEXTURE].Sample(sampler_states[LINEAR], pin.texcoord);
63:     float3 normal_factor = sampled.xyz;
64:     normal_factor = (normal_factor * 2.0) - 1.0;
65:     normal_factor = normalize(normal_factor * float3(m.normal_texture.scale, m.normal_texture.scale, 1.0));
66:     N = normalize((normal_factor.x * T) + (normal_factor.y * B) + (normal_factor.z * N));
67: }
68:
69: float3 diffuse = 0;
70: float3 specular = 0;
71:
72: // Loop for shading process for each light
73: float3 L = normalize(-light_direction.xyz);
74: float3 Li = float3(1.0, 1.0, 1.0); // Radiance of the light
75: const float NoL = max(0.0, dot(N, L));
76: const float NoV = max(0.0, dot(N, V));
77: if (NoL > 0.0 || NoV > 0.0)
```

```
78:  {
79:      const float3 R = reflect(-L, N);
80:      const float3 H = normalize(V + L);
81:
82:      const float NoH = max(0.0, dot(N, H));
83:      const float HoV = max(0.0, dot(H, V));
84:
85:      diffuse += Li * NoL * brdf_lambertian(f0, f90, c_diff, HoV);
86:      specular += Li * NoL * brdf_specular_ggx(f0, f90, alpha_roughness, HoV, NoL, NoV, NoH);
87:  }
88:
89:  float3 emmissive = emmissive_factor;
90:  diffuse = lerp(diffuse, diffuse * occlusion_factor, occlusion_strength);
91:  specular = lerp(specular, specular * occlusion_factor, occlusion_strength);
92:
93:  float3 Lo = diffuse + specular + emmissive;
94:  return float4(Lo, basecolor_factor.a);
95: }
```

2. ビルド・実装し、描画されることを確認する
※モデルデータを「DamagedHelmet」に変更する

【評価項目】

□PBR によるの描画